

Listas Encadeadas

Algoritmos e Programação 2

Prof. Dr. Anderson Bessa da Costa

Universidade Federal de Mato Grosso do Sul

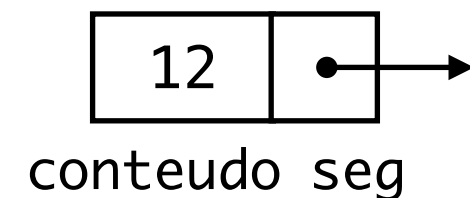
Introdução

- Uma **lista encadeada** é uma representação de uma sequência de objetos na memória do computador
- Cada elemento é armazenado em uma “**célula**” da lista
- As **células** que armazenam elementos consecutivos da sequência não ficam necessariamente em posições consecutivas da memória
- Uma **lista encadeada** é uma sequência de **células**

Definição

- Cada **célula** contém um objeto de determinado tipo e o endereço da célula seguinte (no caso do último célula, esse endereço é **NULL**)
- Suponha que os objetos armazenados nas **células** são números inteiros:

```
struct cel {  
    int conteudo;  
    struct cel *seg;  
};
```



Tipo Celula

- É conveniente tratar as **células** como um novo tipo de dados, que chamaremos **celula**:

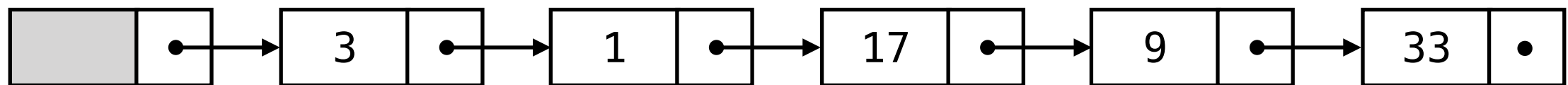
```
typedef struct cel celula;
```

- Uma **celula** c e um ponteiro p para uma **celula** podem ser agora declarados assim:

```
celula c;  
celula *p;
```

Tipo Celula (Cont.)

- Se c é uma célula então $c.\text{conteudo}$ é o conteúdo da célula e $c.\text{seg}$ é o endereço da célula seguinte
- Se p é o endereço de uma célula, então $p \rightarrow \text{conteudo}$ é o conteúdo da célula e $p \rightarrow \text{seg}$ é o endereço da célula seguinte
- Se p é o endereço da última célula da lista então $p \rightarrow \text{seg}$ contém **NULL**



p é uma Lista

- O **endereço** de uma **lista encadeada** é o endereço de sua primeira célula
- Se p é o endereço de uma lista, podemos dizer, simplesmente, “ p é uma lista” e “considere a lista p ”
- Reciprocamente, a expressão “ p é uma lista” deve ser interpretada como “ p é o endereço do primeiro nó de uma lista”
- Observe a natureza recursiva das listas encadeadas: para toda lista encadeada p .
 1. p é NULL ou
 2. $p \rightarrow \text{seg}$ é uma lista encadeada.

Listas Com Cabeça e Sem Cabeça

- Uma **lista encadeada** pode ser vista de duas maneiras:
 - **Com cabeça**
 - Primeira célula sempre existe, mesmo se lista vazia
 - Não armazena conteúdo na cabeça
 - **Sem cabeça**
 - Primeira célula é uma célula ordinária
 - Se lista vazia, não possui nenhuma célula

Listas Com Cabeça

- A primeira célula é a **cabeça** da lista
- Se `lst` é o endereço da cabeça então `lst->seg` vale **NULL** se e somente se a lista está vazia
- Para criar uma lista vazia deste tipo, basta dizer:

```
celula c, *lst;
```

```
c.seg = NULL;  
lst = &c;
```

OU

```
celula *lst;
```

```
lst = (celula *) malloc(sizeof(celula));  
lst->seg = NULL;
```


Listas Sem Cabeça

- Na lista **sem cabeça**, o conteúdo da primeira célula é tão relevante quanto o das demais
- A lista está vazia se não tem célula seguinte
- Para criar uma lista `lst` basta dizer (Trataremos de listas com cabeça nela aula!!)

```
celula *lst;  
lst = NULL;
```

Busca em Lista Encadeada

- É fácil verificar se um objeto x pertence a uma lista encadeada, ou seja, se x é igual ao conteúdo de alguma célula da lista

Busca em Lista Encadeada (2/4)

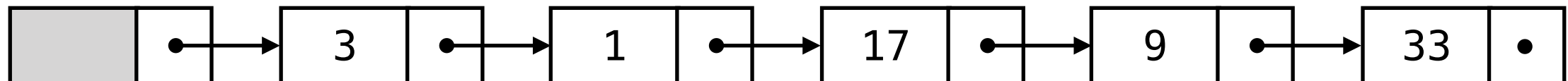
```
1. /* Esta função recebe um inteiro x e uma lista encadeada lst
2. com cabeca. Devolve o endereço de uma célula que contem x ou
3. devolve NULL se tal célula não existe */
4. celula* busque (int x, celula *lst) {
5.     celula *p;
6.
7.     // aponte p para o inicio da lista
8.     p = lst->seg;
9.
10.    // enquanto não cheguei ao fim da lista e não encontrei o x
11.    while (p != NULL && p->conteudo != x)
12.        // aponte p para o elemento seguinte
13.        p = p->seg;
14.
15.    return p;
16.}
```

Busca em Lista Encadeada (3/4)

```
1. /* Esta função recebe um inteiro x e uma lista encadeada lst
2.   com cabeca. Devolve o endereço de uma célula que contem x ou
3.   devolve NULL se tal célula não existe */
4. celula* busqueR (int x, celula * lst) {
5.     if (lst->seg == NULL)
6.         return NULL;
7.     else if (lst->seg->conteudo == x)
8.         return lst->seg;
9.     else
10.        return buscaR(x, lst->seg);
11. }
```

Busca em Lista Encadeada (4/4)

- O que a função retorna se eu buscar pelo elemento 17?
- E pelo 4?



Remoção de Uma Célula

- Suponha que queremos remover uma célula de uma lista
 - Como devemos especificar a célula a ser removido??
- Parece natural apontar para a célula em questão, mas é fácil perceber o defeito da ideia
- Melhor apontar p/ célula anterior ao que queremos remover
 - É bem verdade que esta convenção não permite remover a primeira célula da lista, mas esta operação não é necessária no caso de listas com cabeça

Remoção de Uma Célula (Cont.)

```
1. /* Esta função recebe o endereço p de uma célula em uma
2.  lista encadeada e remove da lista a célula p->seg.
3.  A função supõe que p != NULO e p->seg != NULO */
4. void remove (célula *p) {
5.     célula *lixo;
6.
7.     lixo = p->seg;
8.     p->seg = lixo->seg;
9.     free(lixo);
10. }
```


Inserção de uma Nova Célula

- Suponha que queremos inserir uma nova célula com conteúdo y entre a célula apontado por p e o seguinte
 - É claro que isso só faz sentido se p for diferente de **NULL**

Inserção de uma Nova Célula (Cont.)

```
1. /* A função insere uma nova célula em uma lista encadeada
2.  entre a célula p e o seguinte (supõe-se que p != NULO).
3.  A nova célula terá conteúdo y */
4. void insira (int y, celula *p) {
5.     celula *nova;
6.
7.     nova = (celula *) malloc(sizeof(celula));
8.     nova->conteudo = y;
9.     nova->seg = p->seg;
10.    p->seg = nova;
11.}
```

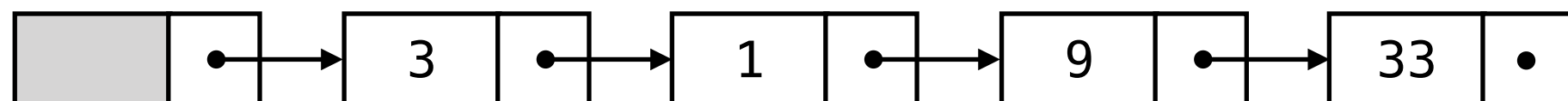
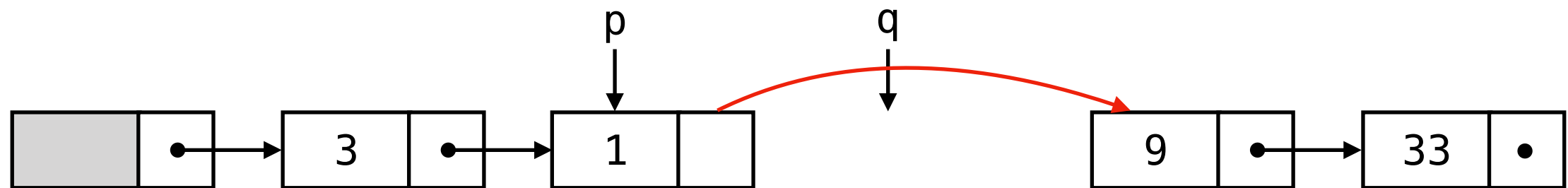
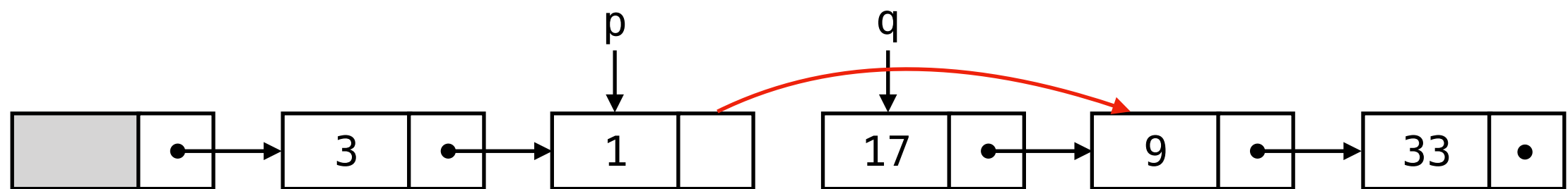
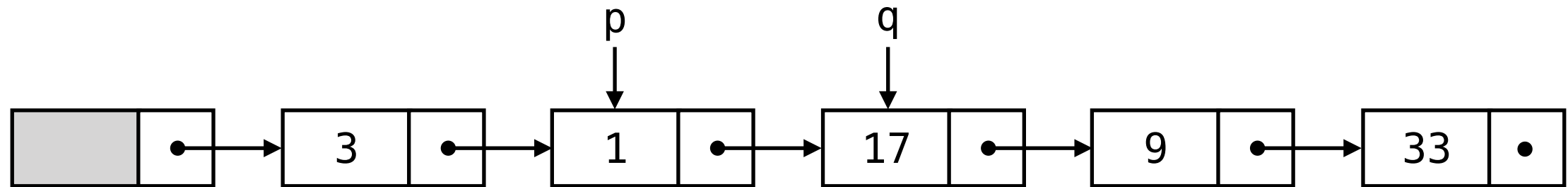
Busca Seguida de Remoção

- Considere uma lista encadeada com cabeça
- Dado um inteiro x , queremos remover da lista a primeira célula que contiver x
 - Se tal célula não existe, não é preciso fazer nada

Busca Seguida de Remoção (2/3)

```
1. /* Esta função recebe uma lista encadeada lst com cabeça e remove
   da lista
2. a primeira célula que contiver x, se tal célula existir */
3. void busque_e_remove (int x, celula *lst) {
4.     celula *q, *p;
5.
6.     p = lst;
7.     q = lst->seg;
8.
9.     while (q != NULL && q->conteudo != x) {
10.        p = q;
11.        q = q->seg;
12.    }
13.
14.    // se encontrou elemento
15.    if (q != NULL) {
16.        p->seg = q->seg;
17.        free(q);
18.    }
19.}
```

Busca Seguida de Remoção (3/3)



Busca Seguida de Inserção

- Suponha agora que queremos inserir uma nova célula com conteúdo y imediatamente antes da primeira célula que tiver conteúdo x
 - Se tal célula não existe, devemos inserir y no fim da lista

Busca Seguida de Inserção (Cont.)

```
1. /* Recebe uma lista encadeada lst com cabeça e insere uma nova
   célula na lista imediatamente antes da primeira que contiver x. Se
   nenhuma célula contiver x, a nova célula será inserida ao final da
   lista. A nova célula terá conteúdo y */
2. void busque_e_insira (int y, int x, celula *lst) {
3.     celula *q, *p, *nova;
4.
5.     nova = (celula *) malloc(sizeof(celula));
6.     nova->conteudo = y;
7.     p = lst;
8.     q = lst->seg;
9.
10.    while (q != NULL && q->conteudo != x) {
11.        p = q;
12.        q = q->seg;
13.    }
14.
15.    nova->seg = q;
16.    p->seg = nova;
17.}
```

Outros Tipos Listas Encadeadas

- Poderíamos definir vários tipos de listas encadeadas além do tipo básico. Seguem dois exemplos importantes:
 - Numa lista encadeada **circular**, a última célula aponta para a primeira. A lista pode ou não ter uma célula-cabeça (Se não tiver cabeça, as expressões “primeira célula” e “última célula” não fazem muito sentido.)
 - Numa lista duplamente **encadeada**, cada célula contém o endereço da célula anterior e o da célula seguinte. A lista pode ou não ter uma célula-cabeça, conforme a conveniência do programador.

Outros Tipos Listas Ligadas (Cont.)

As seguintes questões são apropriadas para qualquer tipo de lista ligada

Em que condições a lista está vazia?

- Como remover o nó apontado por p ? Como remover o nó seguinte ao apontado por p ? Como remover o nó anterior ao apontada por p ? Como inserir um novo nó entre a apontada por p e o anterior? Como inserir um novo nó entre o apontado por p e o seguinte?

Referências

- *FEOFILOFF, P. Algoritmos em Linguagem C, 1. ed. Rio de Janeiro: Elsevier, 2008.
- SZWARCFITER, Jayme Luiz; MARKENZON, Lilian. Estruturas de Dados e seus Algoritmos. Edição: 3a. Editora: LTC. 2010
- CORMEN, T. H.[et al]. Algoritmos: teoria e prática. 3ª ed. Rio de Janeiro: Elsevier, 2012.